Frontiers of Science Awards
for Math/Phys/ISE
[1] *pp.* 1–14

© *International Press*

The Approximation Power of Deep ReLU Neural Networks

I. Daubechies, R. DeVore, S. Foucart, B. Hanin, and G. Petrova[1]

Abstract

The performance of neural networks in targeted applications such as machine learning indicate that they actually possess greater approximation power than traditional methods of nonlinear approximation such as variable knot splines or n-term approximation from dictionaries. The results from [7] summarized here indicate that this is indeed the case. In particular, we showcase examples of large classes of functions which can be efficiently captured by neural networks where classical nonlinear methods fall short of the task. In the original article, we purposefully limited ourselves to the approximation of univariate functions by ReLU networks, but generalizations to functions of several variables or other activation functions have now been obtained. However, even in the simplest of settings initially considered, a theory that completely quantifies the approximation power of neural networks is still desirable.

Contents

1 Introduction **1**

2 Definitions and simple observations **4**

3 Expressiveness of deep ReLU neural networks **5**

4 Approximation by (deep) neural networks **8**

5 Neural network approximation as manifold approximation **11**

1 Introduction

Neural networks produce structured parametric families of functions that have been studied and used for more than 70 years, going back to the work of Hebb in the late 1940's [15] and of Rosenblatt in the 1950's [26]. In the last decades, however, their popularity has surged as they have achieved state-of-the-art performance in a striking variety of machine learning problems, from computer vision [19] to natural language processing [33] and to reinforcement learning and super intelligence. Despite these empirical successes, even their proponents agree that neural networks are not yet well-understood and that a rigorous theory of how and why they work could lead to significant practical improvements [3, 21].

[1]This research was supported by the NSF grants DMS 15-21067 (RD-GP), DMS 18-17603 (RD-GP), DMS 16-22134 (SF), DMS 16-64803 (SF), ONR grants N00014-17-1-2908 (RD), N00014-16-1-2706 (RD), and the Simons Foundation Math + X Investigator Award 400837 (ID).

An often cited theoretical feature of neural networks is that they produce universal function approximators [5,17] in the sense that, given any continuous target function f on a compact domain and a target accuracy $\epsilon > 0$, neural networks with enough judiciously chosen parameters give an approximation to f within an error of size ϵ. Their universal approximation capacity has been known since the 1980's, yet it is not the main reason why neural networks are so effective in practice. Indeed, many other families of functions are universal function approximators, and through the core results of Approximation Theory [8,11], we have a complete understanding of the properties of the target function f which determine how well it can be approximated given a budget for the number of parameters to be used. Such characterizations did not exist for neural network approximation, even in the simplest setting when the target function is univariate and the network's activation function is the **Re**ctified **L**inear **U**nit (ReLU).

The neural networks used in modern machine learning are distinguished from those popular in the 1980's/90's by an emphasis on using *deep* networks (as opposed to shallow networks with one hidden layer). If the universal approximation property were key to the impressive recent successes of neural networks, then the depth of the network would not matter since both shallow and deep networks are universal function approximators.

Our article [7], which we summarize here, focused on the advantages of deep versus shallow architectures in neural networks. Our goal was to put mathematical rigor into the empirical observation that deep networks can approximate many interesting functions more efficiently, per parameter, than shallow networks (see [12, 16, 27, 32, 34, 35] for a selection of rigorous results).

In the few years before [7], there had been a number of interesting papers that address the approximation properties of deep neural networks. Most of them treat ReLU networks since the rectified linear unit is the activation function of preference in many applications, particularly for problems in computer vision. Let us mention, as a short list, some papers most related to our work. It is shown in [13] that deep ReLU networks can approximate functions of d variables as well as linear approximation by algebraic polynomials with a comparable number of parameters. This is done by using the fact (proved by Yarotsky [34]) that power functions x^ν can be approximated with exponential efficiency by deep ReLU networks. Yarotsky also showed that certain classes of classical smoothness (Lipschitz spaces) can be approximated with rates slightly better than that of classical linear methods (see [35]). The main advantage of deep neural networks is that they can output compositions of functions cheaply. This fact has been exploited by many authors (see e.g. [2, 25]).

In the present paper, just as in [7], we address the approximation power of ReLU networks and, in particular, whether such networks are truly more powerful in approximation efficiency than the classical methods of approximation. Although most of the results generalize to the approximation of multivariate functions, we discuss only the univariate setting since this gives us the best chance for definitive results. Our main focus is on the advantages of depth, i.e., what advantages are present in deep networks that do not appear in shallow networks. We restrict ourselves to ReLU networks since they have the simplest structure and should be easiest to understand.

We emphasize that, when discussing approximation efficiency, we assume that f is fully accessible and we ask how well f can be approximated by a neural network with n parameters. This is in contrast to problems of data fitting where, instead of full access to f, we only have some data observations about it. In the latter case, the approximation can only use the given data and its performance would depend on the amount and form of that data. Performance in data fitting is often formulated in a stochastic setting in which it is assumed that the data is randomly generated and both the observations and the gradient descent parameter updates are noisy. The data fitting problem, using a specific form of approximation like neural networks, has two components, commonly referred to as bias and variance. We are concentrating on the bias component. It plays a fundamental role not only in data fitting but also in any numerical procedures based on neural network approximation.

Given two integers $W \geq 2$ and $L \geq 1$, we let (precise definitions are given in the next section)

$$\Upsilon^{W,L} := \{S : \mathbb{R} \to \mathbb{R},\ S \text{ is produced by a ReLU network of width } W \text{ and depth } L\}, \tag{1.1}$$

and denote by $n(W, L)$ the number of its parameters. We fix W and study the approximation families $\Upsilon^{W,L}$ when the number of layers L is allowed to vary. Our interest is in understanding why taking L large, i.e., why using deep networks is beneficial. One way to investigate the approximation power of $\Upsilon^{W,L}$ is to first compare it to known nonlinear approximation families with essentially the same number of degrees of freedom. Since every element in $\Upsilon^{W,L}$ is a **C**ontinuous **P**iece**w**ise **L**inear (CPwL) function, the classical approximation family closest to $\Upsilon^{W,L}$ is the nonlinear set

$$\Sigma_n := \{S : \mathbb{R} \to \mathbb{R},\ S \text{ is a CPwL function with at most } n \text{ distinct breakpoints in } (0, 1)\}.$$

The elements of Σ_n are also called free knot linear splines. We place the restriction that the breakpoints are in $(0, 1)$ because we are concerned with approximation on the interval $[0, 1]$.

When $n \asymp n(W, L)$, the sets Σ_n and $\Upsilon^{W,L}$ have comparable complexity in terms of parameters needed to describe them, since the elements in Σ_n are determined by $2n + 2$ parameters. This comparison also probes the expressive power of depth for ReLU networks because Σ_W is (essentially) the same as the one-layer ReLU network $\Upsilon^{W,1}$, see (2.2).

Several interesting results [6, 23, 32] show that, for arbitrarily large $k \geq 1$ and $n = n(W, L)$ sufficiently large, the set $\Upsilon^{W,L} \setminus \Sigma_{n^k} \neq \emptyset$, cf e.g. [32, Theorem 1.2]. This means that sufficiently deep ReLU networks with n parameters can compute certain CPwL functions whose number of breakpoints exceeds any power of n (the increase of the network depth is necessary as k grows). Note that composing two CPwL functions can multiply the number of breakpoints, allowing networks with L layers of width W to create roughly W^L breakpoints for very special choices of weights and biases, see [32]. By choosing to use the available n parameters in a deep rather than shallow network, one can thus produce functions with many more breakpoints than parameters, albeit these functions have a very special structure.

The first natural question to answer in comparing Σ_n with $\Upsilon^{W,L}$ is whether, for every fixed $W \geq 2$, each function $S \in \Sigma_n$ is in a corresponding set $\Upsilon^{W,L}$ with $n(W, L) \asymp n$, i.e., with a comparable number of parameters. This would guarantee we do not lose anything in terms of expressive power when considering deep networks with fixed width W over shallow networks with fixed depth L. One of our results, Theorem 3.1, gives a resolution to this question and shows that, up to a constant multiplicative factor, fixed-width ReLU networks depending on n parameters are at least as expressive as the free knot linear splines Σ_n. In other words, deep ReLU networks retain all of the approximation power of free knot linear splines but also add something since they can create functions which are far from being in Σ_n. We want to understand the new functions being created and how they can assist us in approximation and thus in data fitting. In this direction, we discuss two classes of functions easily produced by ReLU networks, one consisting of self-similar functions and the other emulating trigonometric functions. Appending these classes to Σ_n naturally provides a powerful dictionary for nonlinear approximation. Similar observation was announced in [12], where the authors noticed that highly oscillatory textures and the Weierstrass function can be exponentially well approximated by sparse ReLU networks.

What types of results could effectively explain the increased approximation power of deep networks as compared with other forms of approximation? One possibility is to exhibit classes K of functions on which the decay rate of approximation error for neural networks is better than for other methods (linear or nonlinear) while depending on the same number of parameters. On this point, let us mention that by now there are several theorems in the literature, see e.g. [2, 4, 24, 28], which show that neural networks perform as well as certain classical methods such as polynomials, wavelets, shearlets, etc. (but they do not show that neural networks perform any better than these methods), or optimally represent certain function classes in terms of Kolmogorov rate-distortion theory, see [2, 12].

We seek more convincing results providing compact classes K that are subsets of Banach spaces X on which neural networks perform significantly better than other methods of approximation. In this direction, we mention at the outset that such sets K cannot be described by classical smoothness (such as Lipschitz, Sobolev, or Besov regularity) because for classical smoothness classes K, there are known lower bounds on the performance for any methods of approximation (linear or nonlinear). These lower bounds are provided by concepts such as entropy and widths. However, let us point out that there is an interesting little twist here that allows deep neural networks to give an improvement over classical approximation methods for certain Lipschitz, Sobolev, and Besov classes. This improvement is possible when the selection of parameters used in the approximation is allowed to be unstable.

2 Definitions and simple observations

We recall that the ReLU function applied to $x = (x_1, \dots, x_d) \in \mathbb{R}^d$, is defined as $\mathrm{ReLU}(x_1, \dots, x_d) := (\mathrm{ReLU}(x_1), \dots, \mathrm{ReLU}(x_d)) := (\max\{0, x_1\}, \dots, \max\{0, x_d\})$.

A fully connected feed-forward ReLU network $\mathcal{N}$ with width W and depth L is a collection of weight matrices $M^{(0)}, \dots, M^{(L)}$ and bias vectors $b^{(0)}, \dots, b^{(L)}$, where $M^{(\ell)}$, $\ell = 1, \dots, L-1$, are of size $W \times W$, $M^{(0)}$ has size $W \times 1$, and $M^{(L)}$ has size $1 \times W$. The biases $b^{(\ell)}$ are vectors of size W if $\ell = 0, \dots, L-1$, and a scalar if $\ell = L$. Each such network $\mathcal{N}$ produces a univariate real-valued function

$$A^{(L)} \circ \mathrm{ReLU} \circ A^{(L-1)} \circ \cdots \circ \mathrm{ReLU} \circ A^{(0)}(x), \quad x \in \mathbb{R},$$

where $A^{(\ell)}(y) = M^{(\ell)} y + b^{(\ell)}$, $\ell = 0, \dots, L$. We then define $\Upsilon^{W,L}$ as the set of such functions resulting from all possible choices of weights and biases. Note that the number of parameters $n(W, L)$ used to generate functions in $\Upsilon^{W,L}$ is

$$c\, W^2 L \le n(W, L) \le C\, W^2 L,$$

where $c, C > 0$ are fixed constants.

Every $S \in \Upsilon^{W,L}$ is a CPwL function on the whole real line. For each input $x := x^{(0)} \in \mathbb{R}$, the value $S(x^{(0)})$ of any $S \in \Upsilon^{W,L}$ is computed after the calculation of a series of intermediate vectors $x^{(\ell)} \in \mathbb{R}^W$, called vectors of activation at layer ℓ, $\ell = 1, \dots, L$, before finally producing the output $x^{(L+1)} = M^{(L)} x^{(L)} + b^{(L)}$.

For example, the *hat function* (also called *triangle function*) $H : [0, 1] \to \mathbb{R}$, defined as

$$H(x) = 2(x-0)_+ - 4\Big(x - \frac{1}{2}\Big)_+ = \begin{bmatrix} 2 & -4 \end{bmatrix} \mathrm{ReLU}\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix} x + \begin{bmatrix} 0 \\ -\frac{1}{2} \end{bmatrix} \right\} = \begin{cases} 2x, & 0 \le x \le \frac{1}{2}, \\ 2(1-x), & \frac{1}{2} < x \le 1, \end{cases} \tag{2.1}$$

belongs to $\Upsilon^{2,1}$.

For $L = 1$, each function in $\Upsilon^{W,1}$ is a CPwL function with at most W breakpoints determined by the nodes in the first layer. Conversely, any CPwL function with $(W-1)$ breakpoints interior to $[0, 1]$, when considered on the interval $[0, 1]$, is the restriction of a function from $\Upsilon^{W,1}$ to that interval. In other words, as functions on $[0, 1]$, we have

$$\Sigma_{W-1} \subset \Upsilon^{W,1} \subset \Sigma_W, \tag{2.2}$$

which means that, for large W, the sets $\Upsilon^{W,1}$ and Σ_W are essentially the same. Therefore, in the one dimensional case, neural networks with one hidden layer have the same approximation power as CPwL functions with the same number of parameters. We refer the reader to [31] for recent results on the approximation power of shallow neural networks in the multidimensional case.

3 Expressiveness of deep ReLU neural networks

In this section, we show that ReLU networks can produce free knot linear splines, self-similar functions, and expansions in Fourier-like Riesz basis of CPwL functions using essentially the same number of parameters that are used to describe these sets. This implies that ReLU networks are at least as expressive as any of these sets of functions. In fact, they are at least as expressive as the union of these sets, which intuitively forms a powerful incoherent dictionary.

3.1 ReLU networks are at least as expressive as free knot linear splines

In this section, we fix $W \geq 4, L \geq 2$, and consider the set $\Upsilon^{W,L}$ of outputs generated by a ReLU neural network of width W and depth L, see (1.1). We claim that $\Sigma_n \subset \Upsilon^{W,L}$, where the number $n(W,L)$ of parameters used to describe $\Upsilon^{W,L}$ is bounded by Cn for a certain fixed constant C.

Theorem 3.1. *Fix a width $W \geq 4$. For every $n \geq 1$, the set Σ_n of free knot linear splines with n breakpoints is contained in the set $\Upsilon^{W,L}$ of functions produced by width-W and depth-L ReLU networks. We have the following values for L and bounds on the number of parameters $n(W,L)$,*

$$L = \begin{cases} 2\left\lceil \frac{n}{q(W-2)} \right\rceil, & n \geq q(W-2), \\ 2, & n < q(W-2), \end{cases} \quad \text{and} \quad n(W,L) \leq \begin{cases} Cn, & n \geq q(W-2), \\ W^2+4W+1, & n < q(W-2), \end{cases}$$

with C an absolute constant. Here $q := \lfloor \frac{W-2}{6} \rfloor$ if $W \geq 8$ and $q := 2$ if $4 \leq W \leq 7$.

In fact, a more general result holds. In order to present this result, we introduce certain neural networks, which we call *special* neural networks. Their output $\overline{\Upsilon}^{W,L}$ is subset $\Upsilon^{W,L}$.

Given a width $W \geq 4$ and depth $L \geq 2$, we focus on networks where a special role is reserved for two nodes in each hidden layer, see the figure below, which depicts these nodes as the first ("top") and at the last ("bottom") node of each hidden layer, respectively. The top neuron (first node), which is ReLU free, is used to simply copy

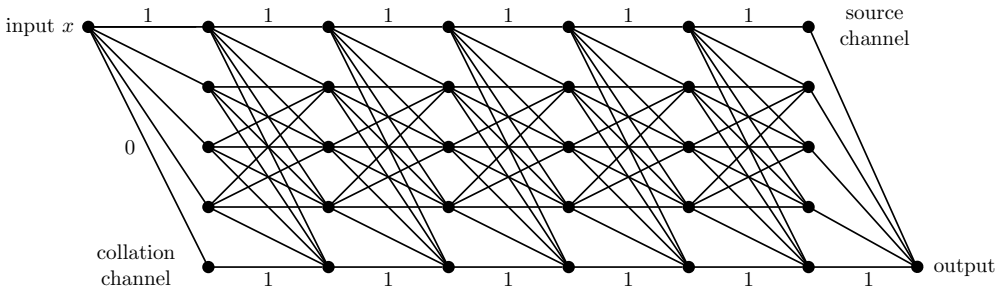


Figure 1: The computation graph associated to $\overline{\Upsilon}^{5,6}$.

the input x. The concatenation of all these top nodes can be viewed as a special "channel" (a term borrowed from the electrical engineering filter-bank literature) that skips computation altogether and just carries x forward. We call this the *source channel* (SC). The bottom neuron (last node) in each layer, which is also ReLU free, is used to collect intermediate results. We call the concatenation of all these bottom nodes the *collation channel* (CC). This channel never feeds forward into subsequent calculations, it only accepts previous calculations. The rest of the channels are *computational channels* (CmC). They consist of neurons (nodes), called computational nodes,

that are equipped with the ReLU function, applied to the input and bias of that node. The fact that a special role is reserved for two channels enforces the natural restriction $W \geq 4$, since we need at least two computational channels. We call these networks (with SC and CC) *special* neural networks, for which we introduce a special notation, featuring a top and a bottom horizontal line to represent the SC and CC, respectively. Namely, we set

$$\overline{\underline{\Upsilon}}^{W,L} = \{S : [0,1] \to \mathbb{R},\ S \ \text{ is produced by a special network of width } W \text{ and depth } L\}.$$

Note that since the SC and CC are ReLU-free, the width-W depth-L special networks do not form a subset of the set of width-W depth-L ReLU networks. However, in terms of sets of functions produced by these networks, the following inclusions hold,

$$\Sigma_n \subset \overline{\underline{\Upsilon}}^{W,L} \subset \Upsilon^{W,L}. \tag{3.1}$$

Before going further, we list some properties of the sets $\Upsilon^{W,L}$ and $\overline{\underline{\Upsilon}}^{W,L}$.

Proposition 3.2. *The following holds*

- *Let $W \geq 2$. For any $\mathcal{Y}_1 \in \Upsilon^{W,L_1}, \dots, \mathcal{Y}_k \in \Upsilon^{W,L_k}$,*
  - *the composition $\mathcal{Y}_k \circ \cdots \circ \mathcal{Y}_1 \in \Upsilon^{W,L}$, where $L = L_1 + \cdots + L_k$,*
  - *the sum $\mathcal{Y}_1 + \cdots + \mathcal{Y}_k \in \overline{\underline{\Upsilon}}^{W+2,L}$, where $L = L_1 + \cdots + L_k$,*
  - *the sum of the $(\mathcal{Y}_i)_+ := \mathrm{ReLU} \circ \mathcal{Y}_i$ satisfies $(\mathcal{Y}_1)_+ + \cdots + (\mathcal{Y}_k)_+ \in \overline{\underline{\Upsilon}}^{W+2,L}$, where the depth $L = k + L_1 + \cdots + L_k$.*
- *Let us denote by $g^{\circ k}$, $k \geq 2$ the function obtained by composing g with itself $k-1$ times.*
  - *If $T \in \Upsilon^{\mathrm{w},L}$, $2 \leq \mathrm{w} \leq W$, then $S = \sum_{i=1}^m a_i T^{\circ i} \in \overline{\underline{\Upsilon}}^{W+2,Lm}$.*
  - *If $T \in \Upsilon^{W_1,\ell}$, $g \in \Upsilon^{W_2,\ell}$, and $W_1 + W_2 = W$, then $S_g = \sum_{i=1}^m a_i g \circ T^{\circ i} \in \overline{\underline{\Upsilon}}^{W+2,\ell(m+1)}$.*

The properties listed in the above Proposition are essential in the proof of the following generalization of Theorem 3.1.

Theorem 3.3. *Consider the functions $S_1 \in \Sigma_{n_1}, \dots, S_k \in \Sigma_{n_k}$ with $n_i \geq (W-2)\lfloor \frac{W-2}{6} \rfloor$, and $W \geq 8$. Their composition*

$$S_k \circ \cdots \circ S_1 \in \Upsilon^{W,L}, \qquad L = 2\sum_{j=1}^{k} \left\lceil \frac{n_j}{\lfloor \frac{W-2}{6} \rfloor (W-2)} \right\rceil, \tag{3.2}$$

where the number of parameters describing $\Upsilon^{W,L}$ satisfies $n(W,L) \leq 34 \sum_{j=1}^k n_j + 2k(W^2+W)$.

Consider the nonconstant functions $S_{i,j} \in \Sigma_{n_{i,j}}$, $i = 1, \dots, m$, $j = 1, \dots, \ell_i$, where we require $n_{i,j} \geq (W-4)\lfloor \frac{W-4}{6} \rfloor$, and $W \geq 10$. Then the sum of their compositions satisfies

$$\sum_{i=1}^{m} a_i S_{i,\ell_i} \circ \cdots \circ S_{i,1} \in \overline{\underline{\Upsilon}}^{W,L} \subset \Upsilon^{W,L}, \quad \text{and} \quad n(W,L) \leq 44 \sum_{i=1}^{m} \sum_{j=1}^{\ell_i} n_{i,j} + 2W(W+1) \sum_{i=1}^{m} \ell_i. \tag{3.3}$$

3.2 ReLU networks efficiently produce functions with self similarity

We have established that ReLU networks can output sums and compositions of CPwL functions. We next show that they can also output CPwL functions with certain self-similar patterns.

Let $0 < \xi_1 < \xi_2 < \cdots < \xi_k < 1$ be a fixed set of breakpoints and let S be any element of the set $S(\xi) := S(0, \xi_1, \dots, \xi_n, 1)$, consisting of all CPwL functions which vanish outside of $[0,1]$ and have breakpoints only at $0, \xi_1, \dots, \xi_n, 1$. We think of S as a *pattern*. It is easy and cheap for ReLU networks to replicate this pattern on many intervals. To describe this, let $\{J_1, \dots, J_m\}$ denote a collection of m intervals contained in $[0,1]$ whose

interiors are pairwise disjoint. We order these intervals from left to right. We say that a CPwL function F is self similar with pattern $S \in S(\xi)$ if

$$F(x) = \sum_{i=1}^{m} S(h_i(x - a_i)), \quad x \in [0,1], \tag{3.4}$$

where $J_i = [a_i, b_i]$ and $h_i = |J_i|^{-1}$, $i = 1, \dots, m$. Thus, the function F consists of a dilated version of S on each of the m intervals J_i. It has roughly km breakpoints but is only described by $2(k+m)$ parameters. In order to produce such a function F, ReLU networks only need a number of parameters of the order $k+m$, and not km as would be naively inferred by regarding F as an element of Σ_{km}. More precisely, the following theorem holds.

Theorem 3.4. *Let $W \geq 8$. Any self-similar function F of the form above with $S \in S(\xi) \subset \Sigma_k$ belongs to $\overline{\Upsilon}^{W,L}$, for a suitable value of L that satisfies $n(W,L) \leq C_1(k+m) + C_2 W^2$ for some absolute constants $C_1, C_2 > 0$.*

3.3 ReLU networks are at least as expressive as Fourier-like sums

In this section, we show that ReLU networks can efficiently produce linear combinations of functions from a certain Riesz basis that emulates the trigonometric basis. The main point to emphasize here is that the linear combinations we consider can involve any of these basis functions not just the first consecutive ones. Approximation by such sums is a classic example of nonlinear approximation.

To describe the Riesz basis we have in mind, we consider the functions $\mathcal{C}, \mathcal{S} : [0,1] \to \mathbb{R}$, given by

$$\mathcal{C}(x) := \begin{cases} 1 - 4x, \, x \in [0, 1/2), \\ 4x - 3, \, x \in [1/2, 1], \end{cases} \qquad \mathcal{S}(x) := \begin{cases} 4x, & x \in [0, 1/4), \\ 2 - 4x, \, x \in [1/4, 3/4), \\ 4x - 4, & x \in [3/4, 1], \end{cases}$$

and $\mathcal{C}_k, \mathcal{S}_k : [0,1] \to \mathbb{R}$, $k \geq 1$, defined for any $x \in [0,1]$ by

$$\mathcal{C}_k(x) := \mathcal{C}(kx - \lfloor kx \rfloor), \qquad \mathcal{S}_k(x) := \mathcal{S}(kx - \lfloor kx \rfloor).$$

The system $\mathcal{F} := (\mathcal{C}_k, \mathcal{S}_k)_{k \geq 1}$ is an important example of a family of CPwL functions which forms a Riesz basis for the set $L_2^0[0,1]$ of square integrable functions on $[0,1]$ with zero mean, namely, there are absolute constants $c, C > 0$ such that, for any two sequences $a, b \in \ell_2(\mathbb{N})$ of real numbers, we have

$$c \sum_{k \geq 1} (a_k^2 + b_k^2) \leq \left\| \sum_{k \geq 1} (a_k \mathcal{C}_k + b_k \mathcal{S}_k) \right\|_{L_2[0,1]}^2 \leq C \sum_{k \geq 1} (a_k^2 + b_k^2). \tag{3.5}$$

The following theorem shows how we can produce via ReLU networks $2k$-term linear combinations of elements from $\mathcal{F}$ with a good control on the depth L.

Theorem 3.5. *Let $W \geq 6$. For every set of indices $\Lambda \subset \mathbb{N}$, the set*

$$\mathcal{F}_\Lambda := \left\{ \sum_{j \in \Lambda} (a_j \mathcal{C}_j + b_j \mathcal{S}_j), \; a_j, b_j \in \mathbb{R}, j \in \Lambda, \;\; |\Lambda| = k \right\} \subset \Upsilon^{W,L},$$

where

$$L = 2 \left\lceil \frac{k}{\lfloor \frac{W-2}{4} \rfloor} \right\rceil (\lceil \log_2(\lambda) \rceil + 2), \qquad \textit{with} \quad \lambda := \max \Lambda,$$

and the wights and biases in this network are bounded by $\max_{j \in \Lambda} \{|a_j|, |b_j|, 8\}$.

4 Approximation by (deep) neural networks

We are more interested in the approximation power of deep neural networks rather than their expressiveness. Of course, these two concepts are closely related, but we now aim at providing convincing results about the approximation power of ReLU networks that establishes their superiority over the existing and more traditional methods of approximation. We shall do so by concentrating on special networks $\overline{\Upsilon}^{W+2,L}$ with a fixed width $W+2$. We introduce the notation

$$\overline{\Upsilon}_0 := \{0\}, \quad \overline{\Upsilon}_m := \overline{\Upsilon}^{W+2,m} \subset \Upsilon^{W+2,m}, \ m \geq 1, \quad \text{and} \quad \overline{\Upsilon} := (\overline{\Upsilon}_m)_{m\geq 0}.$$

The number of parameters determining $\overline{\Upsilon}_m$ is $n(W+2,m) \asymp W^2 m$, and we shall refer to them as *roughly* $W^2 m$. Recall that this nonlinear family possesses the two favorable properties *nestedness* ($\overline{\Upsilon}_{m'} \subset \overline{\Upsilon}_m$ when $m' \leq m$), and the *summation property* ($\overline{\Upsilon}_{m'} + \overline{\Upsilon}_m \subset \overline{\Upsilon}_{m'+m}$).

Let X be any Banach space of (equivalence classes of) functions defined on $[0,1]$. The typical examples of X are the $L_p[0,1]$ spaces, $1 \leq p \leq \infty$, $C[0,1]$, Sobolev and Besov spaces. Our only stipulation on X, at this point, is that it should contain all continuous piecewise linear functions on $[0,1]$. Given $f \in X$, we define its approximation error when using deep neural networks to be

$$\sigma_m(f,\overline{\Upsilon})_X := \inf_{S\in\overline{\Upsilon}_m} \|f-S\|_X, \quad m \geq 0.$$

Since $\overline{\Upsilon}_0 := \{0\}$, we have $\sigma_0(f,\overline{\Upsilon})_X = \|f\|_X$. Given a compact subset $K \subset X$, we define the performance on K to be

$$\sigma_m(K,\overline{\Upsilon})_X := \sup_{f\in K} \sigma_m(f,\overline{\Upsilon})_X, \quad m \geq 0.$$

In a similar way, we define approximation error for other approximation families, in particular $\sigma_m(f,\Sigma)_X$ and $\sigma_m(K,\Sigma)_X$ when $\Sigma := (\Sigma_m)_{m\geq 0}$ is the family of continuous piecewise linear functions. We want to understand the decay rate of $(\sigma_m(f,\overline{\Upsilon})_X)_{m\geq 0}$ for individual functions f and of $(\sigma_m(K,\overline{\Upsilon})_X)_{m\geq 0}$ for compact classes $K \subset X$ and to compare them with the decay rate for other methods of approximation.

4.1 Approximation of classical smoothness spaces

We start this section by revisiting the statement of Theorem 3.1, from where we derive that for $W \geq 4$

$$\Sigma_m \subset \overline{\Upsilon}^{W,2}, \quad 1 \leq m < q(W-2), \quad \text{and} \quad \Sigma_m \subset \Upsilon^{W,\lceil \frac{C}{W^2}\rceil m}, \quad m \geq q(W-2),$$

where $q = 2$ when $4 \leq W \leq 7$, and otherwise $q = \lfloor \frac{W-2}{6} \rfloor$. In addition, for any m we can embed $\Upsilon^{W,m} \subset \overline{\Upsilon}^{W+2,m} = \overline{\Upsilon}_m$ by adding a source and collation channel. So, we can restate these results for any $f \in C[0,1]$ as

$$\sigma_2(f,\overline{\Upsilon})_{C[0,1]} \leq \sigma_m(f,\Sigma)_{C[0,1]}, \qquad 1 \leq m < q(W-2),$$

and

$$\sigma_{\gamma m}(f,\overline{\Upsilon})_{C[0,1]} \leq \sigma_m(f,\Sigma)_{C[0,1]}, \quad m \geq q(W-2), \quad \text{where} \quad \gamma = \gamma(W) := \left\lceil \frac{C}{W^2} \right\rceil.$$

Therefore, all upper bounds for the error of best approximation $\sigma_m(f,\Sigma)_{C[0,1]}$ by the family Σ of free knot linear splines will hold for the error of best approximation $\sigma_{\gamma m}(f,\overline{\Upsilon})_{C[0,1]}$ by the family $\overline{\Upsilon}$. We refer the reader to the paper [8] for a detailed description of free knot spline approximation.

However, the question is can we say more when using deep neural networks in approximating these traditional smoothness classes? The answer is quite surprising and indicative. A series of results beginning with Yarotsky

[34–36] and continuing in [18, 29] show that the approximation rate of some classical smoothness classes, e.g. Lipschitz classes, is dramatically better when using deep networks. Results are now known for approximating the unit ball $U(W^s(L_p[0,1]^d))$, $s>0$, of this Sobolev space with the approximation error measured in $L_p[0,1]^d$ for the same $p\in[1,\infty]$. We limit our discussion to the approximation in the $C[0,1]$ norm of classes like Lip 1 and refer the reader to the above references for the full spectrum of results. The first result in this direction [35] showed that for $W=5$,

$$\sup_{f\in \text{Lip }1}\inf_{S\in\Upsilon^{W,n}}\|f-S\|_{C[0,1]}\le C\frac{|f|_{\text{Lip }1}}{n\ln n}. \tag{4.1}$$

This was a small but still surprising improvement over the optimal rate $O(n^{-1})$ known for approximation by Σ_n. Later, for $W=12$, this was improved to

$$\sup_{f\in \text{Lip }1}\inf_{S\in\Upsilon^{W,n}}\|f-S\|_{C[0,1]}\le C|f|_{\text{Lip }1}n^{-2}. \tag{4.2}$$

This is now known to be the optimal rate which cannot be improved. In the case when $f\in C[0,1]$ is smoother, we have the following theorem.

Theorem 4.1. *For $W\ge 12$ and $f\in W^1(L_p[0,1])$, $1\le p\le\infty$, then*

$$\sigma_m(f,\overline{\Upsilon})_{C[0,1]}\le C(W)\|f'\|_{L_p}m^{-2+1/p},\quad m\ge 2. \tag{4.3}$$

This result was later extended in [30], see Theorem 1 and Theorem 2, to functions from any Sobolev and Besov spaces in any dimension.

4.2 Other function classes exploiting the power of depth

In this subsection, we highlight several other classes of functions whose approximation rates by neural networks far exceed their approximation rates by free knots linear splines or any other standard approximation family. Our constructions are based on variants of the following simple observation.

Proposition 4.2. *For functions $f_k\in\overline{\Upsilon}_k$ satisfying $\|f_k\|_{C[0,1]}=1$ for all $k\ge 1$, and for a sequence $(\beta_k)_{k\ge1}$ in $\ell_1(\mathbb{N})$, the function $F:=\sum_{k\ge1}\beta_k f_k$ has approximation error satisfying*

$$\sigma_{m^2}(F,\overline{\Upsilon})_{C[0,1]}\le\sum_{k>m}|\beta_k|,\quad m\ge 1.$$

When the functions f_k are related to one another, the proposition can be improved by replacing m^2 with a smaller quantity. For example, if $f_k=\phi^{\circ k}$ for a fixed function ϕ in $\Upsilon^{\mathrm{w},\ell}$, with width $2\le \mathrm{w}\le W-2$ and fixed depth ℓ, then Proposition 3.2 reveals that m^2 can be changed to ℓm.

We now present some classes of such functions F that are well approximated by ReLU networks. For the most part, these functions cannot be well approximated by standard approximation families.

The Takagi class of functions: For our first set of examples, let us recall that functions of the form

$$F=\sum_{k\ge1}t^k g\circ\psi^{\circ k},\quad |t|<1, \tag{4.4}$$

with $\psi:[0,1]\to[0,1]$ and $g:[0,1]\to\mathbb{R}$, provide primary examples of self similar functions and dynamical systems [37]. If $g\in\Upsilon^{W_1,\ell}$ and $\psi\in\Upsilon^{W_2,\ell}$, with $W_1+W_2=W$, the partial sum

$$S_m:=\sum_{k=1}^m t^k g\circ\psi^{\circ k}\in\overline{\Upsilon}^{W+2,\ell(m+1)}=\overline{\Upsilon}_{\ell(m+1)}.$$

Therefore, in this case, the function F is approximated by the partial sum S_m with exponential accuracy by ReLU networks,

$$\sigma_{\ell(m+1)}(F,\overline{\Upsilon})_{C[0,1]} \le C\frac{|t|^{m+1}}{1-|t|}, \quad m \ge 1.$$

Now, we consider a special class of functions. For this purpose, we recall that the hat function $H \in \Upsilon^{2,1}$ and its k-fold composition $H^{\circ k} := H \circ H \circ \cdots \circ H$, according to the composition property, belongs to $\overline{\Upsilon}^{2,k}$. On the other hand, the same function $H^{\circ k}$ is in Σ_n only if n is exponential in k. For an absolutely summable sequence $(c_k)_{k\ge 1}$ of real numbers, we consider continuous functions F of the form

$$F := \sum_{k\ge 1} c_k H^{\circ k}.$$

The collection of all such functions is called the Takagi class. It contains a number of interesting and important examples. A good source of information on the Takagi class is [1], from which the two examples below are taken.

For the first example, we take $c_k := 2^{-k}$, which gives the Takagi function

$$T := \sum_{k\ge 1} 2^{-k} H^{\circ k}.$$

Note that we have

$$\sigma_m(T,\overline{\Upsilon})_X \le 2^{-m}, \quad m \ge 1,$$

and so theoretically T can be approximated with exponential accuracy by ReLU networks with roughly $W^2 m$ parameters. In practice, we can approximate it using m parameters. However, T is nowhere differentiable and so it has very little smoothness in the classical sense. This means that all of the traditional methods of approximation will fail miserably to approximate it. Note that the function T has self similarity, in that it satisfies a simple refinement equation.

Other examples take a highly lacunary sequence of coefficients and thereby construct functions in the Takagi class that do not satisfy a Lipschitz condition of any order and yet they can be approximated to exponential accuracy by $\overline{\Upsilon}$. Many functions from the Takagi class are fractals, in the sense that the Hausdorff dimension of their graph is strictly greater than one.

We do not go into the Takagi class more deeply but refer the reader to [1,14] where the properties and applications of the Takagi functions are given as well as numerous examples of similar constructions. The main point to draw from these examples is that the approximation classes A^r for r large contain many functions which are not smooth in any classical sense. This point was also made in [12], where the authors show that oscillatory textures can be approximated with exponential accuracy, see Proposition IX.2, and that, see Proposition IX.3, the Weierstrass function

$$W_{p,a}(x) := \sum_{k=0}^{\infty} p^k \cos(a^k \pi x), \quad p \in (0,1/2), \quad a \in \mathbb{R}^+, \quad ap \ge 1,$$

can be approximated by $S \in \Upsilon^{20,Cn^3}$ with accuracy 2^{-n}, that is $\|W_{a,p} - S\|_{C[-1,1]} \le 2^{-n}$.

Analytic functions: Another example in the Takagi class is the function, see [34],

$$x(1-x) = \sum_{k\ge 1} 4^{-k} H^{\circ k}, \tag{4.5}$$

used as a starting point to show that analytic functions are well approximated by deep neural networks (see [13, 22, 34]), Indeed, it is easily seen that the function x^2 is approximated with exponential accuracy by ReLU networks. From this, one derives that all power functions x^k also are approximated with exponential accuracy.

Then, using the summation property, one concludes that analytic functions and functions in Sobolev spaces are approximated with the same accuracy as their approximation by algebraic polynomials, up to logarithmic factors. Similarly, we can approximate functions on $[0,1]$ from their power series representation. The point we emphasize here is the flexibility of ReLU networks. On one hand, classes of functions with little classical smoothness are well approximated by ReLU networks (some even with exponential accuracy), while, on the other hand, ReLU networks still retain the accuracy of well known approximation methods for classically smooth functions.

5 Neural network approximation as manifold approximation

Up to this point, we have reflected on the expressive power and the corresponding approximation power of deep ReLU networks. In other words, we wondered how well the best approximation from $\overline{\Upsilon}_\ell$ to a target function performs. An important practical issue is the construction of reasonable methods of approximation that yield near-best approximations to any given target function $f \in X$ with e.g. $X = C[0,1]$.

To discuss this problem, we need to formulate what would be considered a reasonable approximation procedure. The set $\overline{\Upsilon}_\ell$ is described by roughly $W^2\ell$ parameters, which are identified by a point in $\mathbb{R}^m$, $m = CW^2\ell$. We let $M = M_m$ be the mapping that sends $z \in \mathbb{R}^m$ to the function $M(z)$ generated by the neural network with the chosen parameters z. We view the collection $\mathcal{M} = \mathcal{M}_{\Updownarrow}$ of all $M(z)$, $z \in \mathbb{R}^m$, as an m-dimensional manifold. Here, in contrast to the usual use of the term manifold in topology, we do not assume a priori any particular smoothness of the mapping M. In this context, we also view any approximation method as providing a mapping $a = a_m : X \to \mathbb{R}^m$ which, for a given $f \in X$, selects the parameters of the network used to approximate f. The approximation to f is then

$$A_m(f) = M_m(a_m(f)), \quad m \geq 0.$$

A fundamental question for both theory and numerical practice is what conditions to impose on a_m and M_m so that the resulting scheme A_m is reasonable. In keeping with the notion of numerical stability, we could require that each of these mappings is a Lip 1 function with a fixed Lipschitz constant Γ independent of m. This means that there is a norm $\|\cdot\|$ on $\mathbb{R}^m$ (typically an ℓ_p norm) such that, for any $f_1, f_2 \in X$,

$$\|a_m(f_1) - a_m(f_2)\| \leq \Gamma \|f_1 - f_2\|_X.$$

The stability of M_m means that, for any $z_1, z_2 \in \mathbb{R}^m$,

$$\|M_m(z_1) - M_m(z_2)\|_X \leq \Gamma \|z_1 - z_2\|.$$

One can lessen the demand on numerical stability to requiring only that the mappings a_m and M_m are continuous, not necessarily Lipschitz. This weaker assumption was used in the definition of manifold widths [9]. This manifold width of a compact set $K \subset X$ is defined as

$$\delta_m(K) := \inf_{(a,M)} \sup_{f \in K} \|f - M(a(f))\|_X,$$

where the infimum is taken over all continuous maps $a : K \to \mathbb{R}^m$ and $M : \mathbb{R}^m \to X$. It is shown in [10] that this milder requirement still puts a restriction on how well sets characterized by classical smoothness can be approximated. For example, if K is the unit ball of Lip α, then $\delta_m(K) \geq Cm^{-\alpha}$. Therefore, the improvements discussed in §4.1 cannot be obtained with continuous selection of parameters. This lack of continuity for some approximation schemes was also recognized in [20]. This may be a crucial point in the framing of results on the instability of certain methods for constructing deep network approximations to target functions from data via optimization methods (such as least squares or constrained least squares methods).

References

[1] P. Allaart and K. Kawamura, The Takagi Function: A Survey, *Real Analysis Exchange*, **37**(1) (2011-2012), 1–54.

[2] H. Bölcskei, P. Grohs, G. Kutyniok, and P. Petersen, Optimal Approximation with Sparsely Connected Deep Neural Networks, *SIAM J. Math. Data Sci,*, **1**(1) (2019), 8–45.

[3] M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheyn., *Geometric deep learning: going beyond Euclidean data*, IEEE Signal Processing Magazine, **34**(4) (2017), 18–42.

[4] C. Chui, X. Li, and H. Mhaskar, Neural networks for localized approximation, *Math. Comp.*, **63** (1994), 607–623.

[5] G. Cybenko, *Approximation by superpositions of a sigmoidal function*, Mathematics of Control, Signals, and Systems (MCSS), **2**(4) (1989), 303–314.

[6] A. Daniely, *Depth separation for neural networks*, Proceedings of Machine Learning Research (COLT), **65** (2017), 690–696.

[7] I. Daubechies, R. DeVore, S. Foucart, B. Hanin, and G. Petrova, Nonlinear approximation and (deep) ReLU networks, Constructive Approximation, **55** (2022), 127–172.

[8] R. DeVore, Nonlinear approximation, *Acta Numer.*, **7** (1998), 51–150.

[9] R. DeVore, R. Howard and C. Micchelli, *Optimal non-linear approximation*, Manuscripta Math., **63** (1989) 469–478.

[10] R. DeVore, G. Kyriazis, D. Leviatan, and V.M. Tikhomirov, *Wavelet compression and nonlinear n-widths*, Advances in Computational Math., **1** (1993) 197–214.

[11] R. DeVore and G. Lorentz, Constructive Approximation, Springer-Verlag, Berlin, 1993.

[12] D. Elbrächter, D. Perekrestenko, P. Grohs, and H. Blöleskei, Deep neural network approximation theory, IEEE Transactions on Information Theory, **67** (2021), 2581–2623.

[13] W. E and Q. Wang, Exponential Convergence of the Deep Neural Network Approximation for Analytic Functions, Sci. China Math., **61** (2018), 1733–1740.

[14] M. Hata, Fractals in Mathematics, *Patterns and Waves-Qualitative Analysis of Nonlinear Differential Equations*, (1986), 259–278.

[15] D. Hebb, The organization of behavior: A neuropsychological theory, *Wiley*, (1949).

[16] B. Hanin and M. Sellke, Approximating continuous functions by ReLU nets of minimal width, arXiv:1710.11278.

[17] K. Hornik, M. Stinchcombe, and H. White, *Multilayer feedforward networks are universal approximators*, Neural networks, **2**(5) (1989), 359–366.

[18] Jianfeng Lu, Zuowei Shen, Haizhao Yang, and Shijun Zhang, Deep network approximation for smooth functions,

[19] A. Krizhevsky, I. Sutskever, and G. Hinton, Imagenet classification with deep convolutional neural networks, *NIPS* (2012).

[20] P. Kainen, V. Kurkova, and A. Vogt, *Approximation by neural networks is not continuous*, Neurocomputing, **29** (1999), 47–56. I.

[21] Y. LeCun, Y. Bengio, and G. Hinto., *Deep learning*, Nature, **521**(7553) (2015), 436.

[22] S. Liang and R. Srikant, Why Deep Neural Networks for Function Approximation?, arXiv:1610.04161

[23] M. Mehrabi, A. Tchamkerten, and M. Yousefi, Bounds on the Approximation Power of Feedforward Neural Networks, *ICML*, (2018).

[24] H. N. Mhaskar and T. Poggio, Deep vs. shallow networks: An approximation theory perspective, *Analysis and Applications*, **14** (2016), 829–848.

[25] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao, Why and when can deep-but dot shallow-networks avoid the curse of dimensionality: a review, *International Journal of Automation and Computing*, **14**(5) (2017), 503–519.

[26] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychological review*, **65**(6) (1958), 386.
[27] I. Safran and O. Shamir, Depth-width tradeoffs in approximating natural functions with neural networks, arXiv:1610.09887.
[28] U. Shaham, A. Cloninger, and R. Coifman, Provable approximation properties for deep neural networks, *Applied and Computational Harmonic Analysis*, **44**(3) (2018), 537–557.
[29] Z. Shen, H. Yang, and S. Zhang, Deep Network Approximation Characterized by Number of Neurons, arXiv:1906.05497.
[30] J. W. Siegel, Optimal Approximation Rates for Deep ReLU Neural Networks on Sobolev and Besov Spaces, *JMLR*, **24** (2023), 1–52.
[31] J. W. Siegel, and J. Xu, Sharp bounds on the approximation rates, metric entropy, and n-widths of shallow neural networks, *Foundations of Computational Mathematics*, **24(2)** (2024), 481–537.
[32] M. Telgarsky, Representation benefits of deep feedforward networks, arXiv:1509.08101.
[33] Y. Wu, M. Schuster, Z. Chen, Q. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al., Google's neural machine translation system: Bridging the gap between human and machine translation, arXiv:1609.08144.
[34] D. Yarotsky, Error bounds for approximations with deep ReLU networks, Neural Networks, **94** (2017), 103–114.
[35] D. Yarotsky, Quantified advantage of discontinuous weight selection in approximations with deep neural networks, arXiv:1705.01365.
[36] D. Yarotsky, Optimal approximation of continuous functions by very deep ReLU networks, In S. Bubeck, V. Perchet, and P. Rigollet, editors, Proceedings of the 31st Conference On Learning Theory, volume 75 of Proceedings of Machine Learning Research, 639–649. PMLR, 06-09 Jul 2018, arXiv:1802.03620.
[37] M. Yamaguti and M. Hata, *Weirstrass's function and Chaos*, Hokkaido. Math. J. **12** (1983), 333–342.

Department of Mathematics, Duke University
Durham, USA.
E-mail address: ingrid@math.duke.edu

Department of Mathematics, Texas A&M University
College Station, USA.
E-mail address: ronalddevore@tamu.edu,foucart@tamu.edu,gpetrova@tamu.edu

Department of Operations Research & Financial Engineering, Princeton University
Princeton, USA.
E-mail address: bhanin@princeton.edu